

AERO21111 Space Systems - MATLAB Tutorial 1

Propulsion Sensitivity and Mission Analysis

In the next five tutorials, this LiveScript will cover performance characteristics and requirements for the Sentinel-6B mission. The aim is to apply knowledge from what is taught in this module to a real life problem and to analyse propulsion sensitivity, thermal design, communications link budget, attitude determination and control and power and mass budgets in order to enhance my understanding of the various systems on board a spacecraft.

```
clear all
```

Task 1 - Identifying and rearranging fuel mass equation

On page 107 of the course notes, equation 5.30 states:

$$\Delta v = I_{sp} g \ln \frac{m_0}{m_1} - \frac{D}{m} \Delta t - g \sin \gamma \Delta t$$

This equation is written more simply in the course notes as equation 5.31:

$$\Delta v = \Delta v_{ideal} - \Delta v_D - \Delta v_g$$

The term Δv_D corresponds to atmospheric drag loss and the term Δv_g corresponds to the loss due to gravity. In task 1, the tutorial states that gravitational effects are negligible and the manoeuvre will be completed whilst already in orbit, therefore both of these terms are 0.

Removing these terms and equating equations 5.30 and 5.31, the total change in velocity required for the Sentinel-6B orbital manoeuvre is:

$$\Delta v = \Delta v_{ideal} = I_{sp} g \ln \frac{m_0}{m_1}$$

The values given in the initial problem description are:

$I_{sp} = 200s - 300s$, $\Delta v = 1 - 2km/s$ and $m_1 = m_{dry} = 1000 kg$, where m_1 is the dry mass (m_{dry}) of the spacecraft after the completion of the manoeuvre, I_{sp} is the specific impulse range and Δv is the range of required velocity changes.

Acceleration due to Earth's gravity is:

$$g = 9.81 m/s^2$$

The initial mass of the spacecraft before the manoeuvre begins is the sum of the dry mass and fuel mass:

$$m_0 = m_{dry} + m_{fuel}$$

Inserting all of these variables into equation 5.31 gives the Δv in terms of dry mass and fuel mass:

$$\Delta v = I_{sp} g \ln \frac{m_{dry} + m_{fuel}}{m_{dry}}$$

Rearranging this to make m_{fuel} the subject of the formula gives the fuel mass equation required for task 1:

$$m_{fuel} = m_{dry} \left(e^{\frac{\Delta v}{I_{sp} g}} - 1 \right)$$

Task 2 - Establishing input values

From the equation developed from task 1, there are two known input values, meaning I can create constants in my code for the dry mass and the acceleration due to Earth's gravity:

```
m_d      = 1000;           % Dry mass of the satellite [kg]
g_0      = 9.81*10^(-3);   % Acceleration due to gravity [km/s^2]
```

There are two more input variables to include in the code: Δv and I_{sp} . By constructing two vectors, I can cover the range of values given within the problem. I have chosen to use a colon method instead of using the linspace command:

```
delta_v = 1:0.02:3;        % Required velocity change [km/s]
I_sp    = 200:1:300;       % Specific impulse [s]
```

This has created an array for each variable for the given range, with increments of 0.02 km/s for change in velocity and increments of 1s for specific impulse. I chose these increments so that the arrays produced would be the same size

Task 3 - Converting vectors into a 2D mesh

The meshgrid command can be used to convert the two vectors above into equally sized arrays and plot them in a 2D mesh, which will allow them to be compared and analysed.

```
x      = I_sp;             % Defines x axis of mesh
y      = delta_v;          % Defines y axis of mesh
[X,Y]  = meshgrid(x,y);    % Creates an equally sized array for I_sp and
                           % delta_v ranges
```

Task 4 - Calculating mass of fuel required

All defined inputs can now be substituted into the equation for mass of fuel required from task 1:

```
m_f      = m_d*(exp(Y./(X.*g_0)) - 1); % Fuel mass required for a range of I_sp
                                         % and delta_v inputs [kg]
```

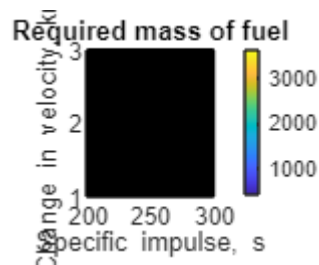
Task 5 - Producing a surface plot

In order to conduct the required sensitivity analysis, I can use the surf and view commands to create a surface plot, which presents the plotted variables (I_{sp} and Δv) in a 2D perspective:

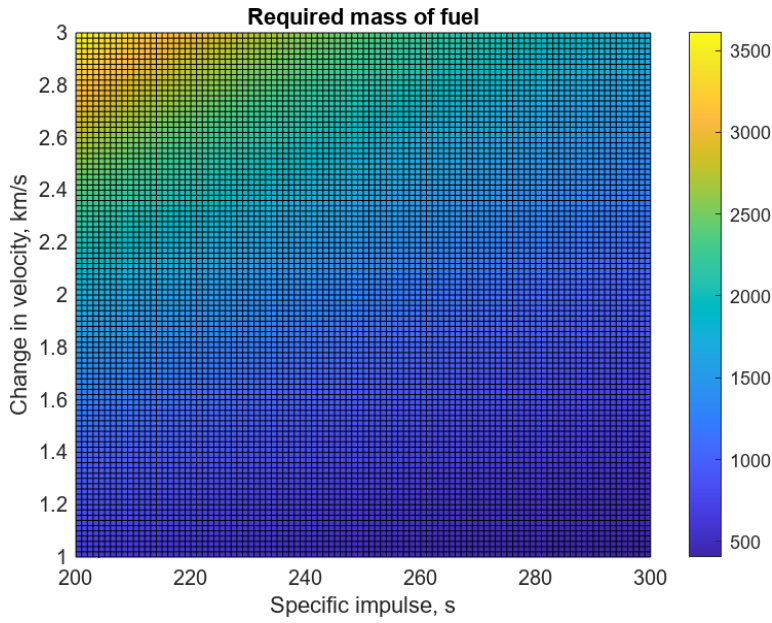
```
surf(X,Y,m_f) % Creates a surface plot in terms of I_sp, delta_v and m_f
view(2)       % Displays the plot in 2D
```

To provide an effective tool to the mission design team, the surface plot created should include a clear title, labels and a colourbar. This should make the sensitivity analysis easier.

```
title('Required mass of fuel') % Adds an appropriate title
xlabel('Specific impulse, {s}') % Adds the required label to the x axis
ylabel('Change in velocity, {km/s}') % Adds the required label to the y axis
colorbar % Generates a colourbar to show mass of
```



```
% fuel required [kg]
```



This plot can now be used to identify the fuel mass required for Sentinel-6B to complete a manoeuvre, given known values of specific impulse and change in velocity.

Task 6 - Identifying mass of fuel required

Using the surface plot created in task 5 and the values given in the problem description ($I_{sp} = 230s$ and $\Delta v = 1.3 \text{ km/s}$), I can now identify the mass of fuel require in kg to perform the manoeuvre:

$$m_{fuel} = 779.201 \text{ kg}$$

Task 7 - Identifying a suitable launch vehicle

- **Maximum spacecraft mass assessment:**

The total launch mass of this spacecraft is:

$$m_{total} = m_{dry} + m_{fuel} = 1000 + 779.201 = 1779.201 \text{ kg},$$

as m_{dry} was given as 1000 kg in the tutorial. Since this is only an estimate in the early stage of this analysis, this value of total launch mass for Sentinel-6B can be rounded to 2 significant figures, giving:

$$m_{total} = 1800 \text{ kg}$$

Table 1 shows that m_{total} is much less than all four values of maximum spacecraft mass, therefore all four rockets meet this requirement.

- **Reliability assessment:**

Using the values given in Table 1, I can calculate the launch reliability for each rocket:

Falcon 9 v1.2 = 98.39%, Ariane 5 = 95.45%, Atlas V 551 = 98.85%, Delta IV Medium = 94.87%.

Both Ariane 5 and Delta IV Medium have a reliability of below 98%, therefore I will not be choosing these two rockets as an option because the ability to rely on a successful launch is extremely important.

- **Greenhouse gas emission assessment:**

Comparing carbon dioxide and water vapour emissions from Table 1 for the remaining two rockets, it is clear that for both sets of emissions, the Atlas V 551 emits much less tonnes of GHGs than the Falcon 9 v1.2, meaning it is the least polluting out of these two rockets.

- **Launch site assessment:**

All launch latitudes are below 66° , meaning technically, all of them could be suitable for this satellite since they will all have two launch windows. However, the closer the launch site is to the equator, the more efficient the launch will be. This is because the equator moves faster than the Earth's poles due to the Earth being oblate. This characteristic means that a launch closer to the equator will provide a larger initial velocity compared to when launching from/near the poles and this is ideal because less fuel will be required for the launch to reach the velocities required. The launch site with the lowest latitude (and therefore closer to the equator) is Guiana Space Centre (FRGUI) in French Guiana.

- **Final launch vehicle and site selection:**

After assessing mass, reliability, emissions and launch sites, using Table 1 and Table 2, I have chosen a rocket and launch site that meet the criteria the best overall. The final selection is the Atlas V 551 to be launch from Guiana Space Centre.

AERO21111 Space Systems - MATLAB Tutorial 2

Thermal Design

Task 1 - Defining inputs and physical constants

Most values have been given in Tables 1 & 2 from the second tutorial, however, for the mass of the spacecraft I will use the value I calculated in Task 7 of the previous tutorial ($m_{total} = 1800 \text{ kg}$). I am assuming that Sentinel-6B is a cube, meaning I must also calculate the total surface area that can absorb/reflect radiation (ie two faces of the cube), given that one face is 3.35 m^2 :

$$A_{total} = 3.35 * 2 = 6.7 \text{ m}^2$$

I can now input all of these values into my code:

```
q_s      = 1400;           % Solar radiation flux [W*m^-2]
q_e      = 240;           % Earth radiation flux [W*m^-2]
a_r      = 0.3;           % Earth albedo
sigma    = 5.67*10^-8;    % Stephan-Boltzmann constant [W*m^-2*K^-4]
R_E      = 6371*10^3;     % Radius of Earth [m]
mu       = 3.986*10^14;   % Standard gravitational parameter of Earth [m^3*s^-2]
h        = 1336*10^3;     % Altitude [m]
alpha_s  = 0.46;         % Absorbtivity
epsilon  = 0.77;         % Emittance
m        = 1800;         % Mass [kg]
P_sun    = 1000;         % Power dissipated by spacecraft in sunlight [W]
P_eclipse = 500;         % Power dissipated by spacecraft in eclipse [W]
A_surf   = 3.35;         % Surface area of single face [m^2]
A_total  = 6.7;          % Total surface area of spacecraft [m^2]
c_p      = 900;          % Specific heat capacity [J*kg^-1*K^-1]
T_i      = 263;          % Initial temperature [K]
```

Task 2 - Calculating the thermal time constant

The thermal time constant is given in the course notes on page 137 as equation 6.45:

$$\tau_t = \frac{mc_p}{\epsilon\sigma A_{total}}$$

Implementing this into my code:

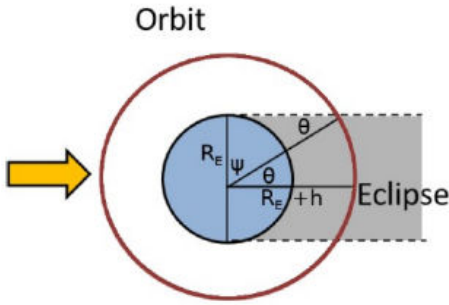
```
tau_t = (m*c_p)/(epsilon*sigma*A_surf*2);           % Thermal time constant
```

Task 3 - Calculating eclipse duration

Assuming the beam of light from the Sun is normal to the surface, the orbit of Sentinel-6B is circular and therefore has a constant velocity, we can use Equation 1 to calculate the angle, θ , which corresponds to half of the distance spent in the eclipse section:

$$\theta = \arcsin\left(\frac{R_e}{R_e + h}\right)$$

This angle is more clearly shown in the following image, given as Figure 1 in the tutorial:



Using the parameters defined in task 1 of this code, I can calculate the value of θ in degrees:

```
theta = asind(R_E/(R_E + h)); % Half of angle at which the satellite
                                % will be in eclipse [deg]
```

In order to find the time spent in the eclipse, I must multiply the orbital period by the fraction of the total distance of each orbit spent in eclipse. The orbital period of a satellite is given by Equation 3.9 on page 44 of the course notes:

$$\tau = 2\pi \sqrt{\frac{r^3}{\mu}}$$

We can now use these equations and inputs to calculate t_e in seconds:

```
T_o = 2*pi*(sqrt(((R_E + h)^3)/mu)); % Orbital period [s]
theta_e = (2*theta)/360; % Eclipse angle fraction
t_e = theta_e*T_o; % Eclipse duration for satellite [s]
```

Task 4 - Calculating equilibrium temperatures

The equations for thermal power input, Q_{in} , and equilibrium temperature, T_{eq} , during Sun exposure are given in the course notes on page 136 as Equations 6.37 and 6.40 respectively:

$$Q_{in} = q_s \alpha_s A_{s/c}^S + a_r q_s \alpha_s F_{12} A_{s/c}^P \cos \phi \cos \theta_s + q_{\oplus} \epsilon F_{12} A_{s/c}^P + P_{dis} \quad \text{and} \quad T_{eq} = \sqrt[4]{\frac{Q_{in}}{\epsilon \sigma A_{total}}}$$

Since we are assuming that the point of this calculation is when the satellite is directly between the Sun and Earth, on a principal axis going through the centres of both, $\phi = 0$ and $\theta = 0$, meaning $\cos \phi = 1$ and $\cos \theta = 1$. The view factor (F_{12}) and radius ratio (R_R) are given in the course notes on page 132 as Equations 6.22 and 6.23 respectively:

$$F_{12} = \frac{\cos \theta}{(1 + R_R)^2} \quad \text{and} \quad R_R = \frac{h}{R_{\oplus}}$$

The equations for these two values during eclipse exposure are given in the tutorial as Equations 2 and 3:

$$Q_{in} = q_{EAEC} \quad \text{and} \quad T_{eq} = \sqrt[4]{\frac{Q_{in}}{\epsilon \sigma A_{total}}}$$

Translating these six equations into code gives:

```
phi = 0; % Angle as measured for a co-planar
          % orbit
theta_s = 0; % Angle between orbit plane and
              % Earth-Sun vector

R_R = h/R_E; % Radius ratio
F_12 = (cos(theta))/((1+R_R)^2); % View factor
Q_in_S = q_s*alpha_s*A_surf + a_r*q_s*alpha_s*F_12*A_surf*cos(phi)*cos(theta_s) + ...
          q_e*epsilon*F_12*A_surf + P_sun + P_eclipse; % Thermal power input during Sun
                                                         % exposure [W]
T_eq_S = (Q_in_S/(epsilon*sigma*A_total))^(1/4); % Equilibrium temperature during Sun
                                                  % exposure [K]

Q_in_E = q_e*A_surf*epsilon; % Thermal power input during eclipse
                              % exposure [W]
T_eq_E = (Q_in_E/(epsilon*sigma*A_total))^(1/4); % Equilibrium temperature during
                                                  % eclipse exposure [K]
```

Task 5 - Calculating change in temperature over time

This task uses Equation 4 given in the tutorial to calculate the change in temperature for a given time step:

$$\frac{dT}{dt} = \frac{T_{eq}^4 - T^4}{\tau_t}$$

In order to show the trend of this model more clearly, I will show the change in temperature over 5 orbits, which will be calculated using a while loop. An if statement will also be used to model the change of thermal conditions, depending on whether the spacecraft is in eclipse or not. I am assuming that each orbit starts immediately after the eclipse phase.

```
P = 5*T_o; % Sets the total simulation
            % time to 5 full orbits [s]
t_s = 0; % Sets the current simulation
          % time to start at 0 [s]
t = []; % Creates an array for the
        % time elapsed
counter = 0; % Uses a counter variable to
             % save temperature and time
             % values into an array
T = T_i; % Sets the temperature to
          % start at the initial
          % temperature defined in task
```



```

t_o = 0; % 1 [K]
dT = 0; % Sets the current orbit time
        % to start at 0 [s]

while t_s <= P % Checks that the current
               % simulation time has not
               % exceeded the total
               % simulation time

    counter = counter + 1;
    t(counter) = t_s; % Saves current simulation
                     % time
    T_sc(counter) = T; % Saves current spacecraft
                       % temperature
    if t_o < T_o - t_e % Checks if spacecraft is in
                       % Sun exposure
        dT = (((T_eq_S^4) - (T^4))/tau_t)*((T_o - t_e)/5); % Uses thermal equilibrium
        t_o = t_o + ((T_o - t_e)/5); % temperature for Sun exposure
        t_s = t_s + ((T_o - t_e)/5); % Adds the change in time to a
                                     % single orbit
    elseif t_o >= T_o - t_e && t_o < T_o % Adds the change in time to
                                         % the current simulation time
        dT = (((T_eq_E^4) - (T^4))/tau_t)*(t_e/5); % Checks if spacecraft is in
        t_o = t_o + (t_e/5); % eclipse
        t_s = t_s + (t_e/5); % Uses equilibrium temperature
    elseif t_o >= T_o % for eclipse
        % Checks if current orbit time
        % is greater than one orbital
        % period
        t_o = 0; % Restarts loop and resets
                 % current orbit time
    end
    T = T + dT; % Adds change in temperature
               % before the loop restarts
end

```

Task 6 - Plotting temperature vs time

Spacecraft temperature can now be plotted against the total simulation time using the plot function to show the change in temperature of Sentinel-6B as it orbits the Earth for five complete orbits:

```

figure
hold on
plot(t, T_sc-273.15); % Converts current spacecraft

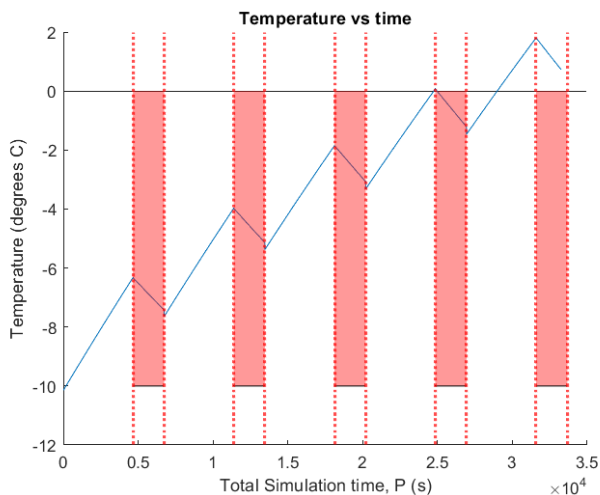
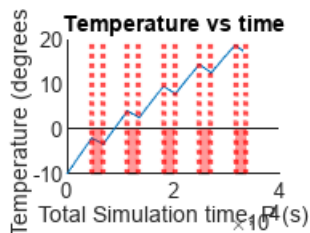
```

```
% temperature from Kelvin to
% degrees Celsius
```

```
title("Temperature vs time");
xlabel("Total Simulation time, P (s)");
ylabel("Temperature (degrees C)");
```

In order to display on the graph the eclipse phases in each orbit, I have decided to shade the area between each pair of two red dotted lines:

```
for i = 1:5 % Creates loop for all five
             % orbits
    xline(t_o*i - t_e, LineWidth=2, LineStyle=":", Color='r'); % Initial line identifying
                                                             % start of eclipse for that
                                                             % specific orbit
    xline(t_o*i, LineWidth=2, LineStyle=":", Color='r'); % Line identifying end of
                                                             % eclipse for that specific
                                                             % orbit
    ha = area([t_o*i-t_e t_o*i],[-10 -10],'FaceColor','r'); % Shades region between lines
                                                             % and indicates eclipse phase
    alpha(.4); % Makes shaded area
                                                             % transparent
end
hold off
```



Discussion

Looking at the graph produced in task 6, it meets the temperature requirements of -15°C to 30°C for the first few orbits, however, it is clear that since the temperature during Sun exposure will continue to increase rapidly, after several more orbits, the temperature of the spacecraft will exceed 30°C , meaning this will eventually lead to component failure. In order to implement effective control measures to keep the temperature within the given range, I have decided to change the absorbtivity of the spacecraft coating material in order to decrease the temperature change during Sun exposure. As seen in the equations outlined in task 4, this change will not affect the equilibrium temperature during eclipse exposure.

In order to achieve a new graph showing the temperature requirements have been met, tasks 4, 5 and 6 should be repeated with a new value of absorbtivity, $\alpha_s = 0.28$. I found this value by trial and error, I ran my code with different values of α_s until the graph showed the intended trend.

```
alpha_s_new = 0.28; % Defines new value of
                    % absorbtivity

phi = 0; % Angle as measured
          % for a co-planar
          % orbit

theta_s = 0; % Angle between orbit
              % plane and Earth-Sun
              % vector

R_R = h/R_E; % Radius ratio
F_12 = (cos(theta))/((1+R_R)^2); % View factor
Q_in_S_new = q_s*alpha_s_new*A_surf + a_r*q_s*alpha_s_new*F_12*A_surf*cos(phi)*...
            cos(theta_s) + q_e*epsilon*F_12*A_surf + P_sun + P_eclipse; % New thermal power
                                                                    % input during Sun
                                                                    % exposure [W]
T_eq_S_new = (Q_in_S_new/(epsilon*sigma*A_total))^(1/4); % New equilibrium
                                                                    % temperature during
                                                                    % Sun exposure [K]

Q_in_E = q_e*A_surf*epsilon; % Thermal power input
                              % during eclipse
                              % exposure [W]
T_eq_E = (Q_in_E/(epsilon*sigma*A_total))^(1/4); % Equilibrium
                                                                    % temperature during
                                                                    % eclipse exposure [K]

P = 5*T_o; % Sets the total
           % simulation time to
           % 5 full orbits [s]

t_s = 0; % Sets the current
         % simulation time to
         % start at 0 [s]

t = []; % Creates an array for
        % the time elapsed

counter = 0; % Uses a counter
```

```

T = T_i;

t_o = 0;

dT_new = 0;

while t_s <= P

    counter = counter + 1;
    t(counter) = t_s;

    T_sc_new(counter) = T;

    if t_o < T_o - t_e

        dT_new = (((T_eq_S_new^4) - (T^4))/tau_t)*((T_o - t_e)/5);

        t_o = t_o + ((T_o - t_e)/5);

        t_s = t_s + ((T_o - t_e)/5);

    elseif t_o >= T_o - t_e && t_o < T_o

        dT_new = (((T_eq_E^4) - (T^4))/tau_t)*(t_e/5);

        t_o = t_o + (t_e/5);
        t_s = t_s + (t_e/5);
    elseif t_o >= T_o

        t_o = 0;

```

% variable to save
 % temperature and time
 % values into an array
 % Sets the temperature
 % to start at the
 % initial temperature
 % defined in task 1
 % [K]
 % Sets the current
 % orbit time to start
 % at 0 [s]
 % Sets the new change
 % in temperature to
 % start at 0

 % Checks that the
 % current simulation
 % time has not
 % exceeded the total
 % simulation time

 % Saves current
 % simulation time
 % Saves new current
 % spacecraft
 % temperature
 % Checks if spacecraft
 % is in Sun exposure
 % Uses new thermal
 % equilibrium
 % temperature for Sun
 % exposure
 % Adds the change in
 % time to a single
 % orbit
 % Adds the change in
 % time to the current
 % simulation time
 % Checks if spacecraft
 % is in eclipse
 % Uses equilibrium
 % temperature for
 % eclipse

 % Checks if current
 % orbit time is
 % greater than one
 % orbital period
 % Restarts loop and

```

end
T = T + dT_new;

end

figure
hold on
plot(t, T_sc_new-273.15);

title("Temperature vs time");
xlabel("Total Simulation time, P (s)");
ylabel("Temperature (degrees C)");
for i = 1:5

xline(t_o*i - t_e, LineWidth=2, LineStyle=":", Color='r');

xline(t_o*i, LineWidth=2, LineStyle=":", Color='r');

ha = area([t_o*i-t_e t_o*i],[-10 -10],'FaceColor','r');

alpha(.4);

end
hold off

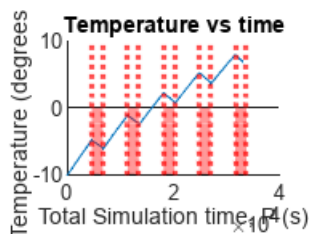
```

% resets current orbit
% time

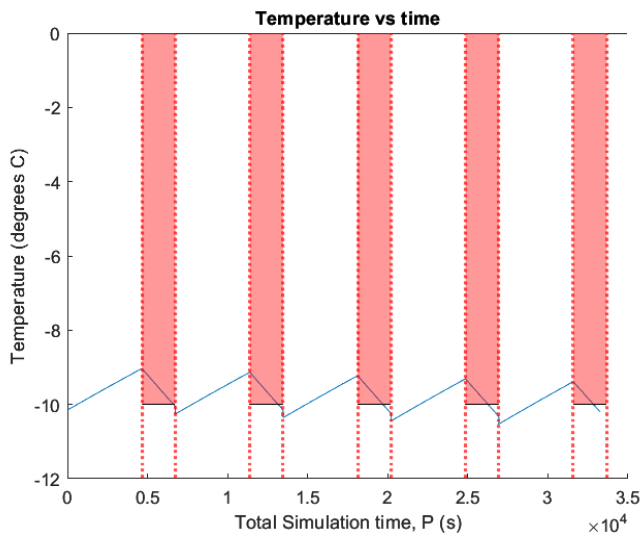
% Adds new change in
% temperature before
% the loop restarts

% Converts new current
% spacecraft
% temperature from
% Kelvin to degrees
% Celsius

% Creates loop for all
% five orbits
% Initial line
% identifying start of
% eclipse for that
% specific orbit
% Line identifying end
% of eclipse for that
% specific orbit
% Shades region
% between lines and
% indicates eclipse
% phase
% Makes shaded area
% transparent



Looking at the second graph of temperature against time produced with a new value of absorbtivity, it is clear that the spacecraft will now always be at the required temperature for all of its components because after each orbit, the temperature goes back to the same value.



AERO21111 Space Systems - MATLAB Tutorial 3

Communications Link Budget Analysis

Task 1 - Identifying transmitter and receiver

In the tutorial, Table 2 gives information about a downlink. Since the platform communications link is a downlink, this means that the ground station is the receiving station and the space segment is the transmitting station.

Task 2 - Defining constants

Inputting the values given in tables 1, 2 and 3 into my code:

```
P_Tgs      = 10*log10(35);    % Ground station transmission power [dBW]
T_Rgs      = 257;            % Ground station noise temperature [K]
D_gs       = 4.5;            % Antenna diameter [m]

BER         = 8*10^-6;        % Bit Error Rate
R_b         = 150*10^6;        % Transmission rate [bps]
v_d         = 8.9;            % Downlink frequency [GHz]
L_i         = 1.85;           % Implementation losses [dB]
L_add       = 7;              % Additional losses [dB]

P_Tss       = [30, 90, 54];   % Transmission power array for space segment [W]
T_Rss       = [156, 160, 170]; % Noise temperature array for space segment [K]
theta_bw    = [35, 30, 33];   % Antenna beamwidth array for space segment [degrees]
m_ss        = [30, 12, 2];     % Mass array for space segment [kg]
```

I have defined the properties of the space segment as arrays, with (1) being the parabolic antenna, (2) being the helix antenna and (3) being the horn antenna. I have also converted some constants into their respective decibel terms to make them easier for later use.

Task 3 - Identifying antenna gain equations

Using the information given in Table 3 of the tutorial and the equations on page 178 of the course notes, I can identify the equations required to calculate antenna gain:

- Parabolic antenna

$$G = 44.3 - 10 \log(\theta^2)$$

- Helix antenna

$$G = 10.3 + 10 \log\left(\frac{\pi^2 D^2 L}{\lambda^3}\right) \quad \text{and} \quad \theta = \frac{52}{\sqrt{\frac{\pi^2 D^2 L}{\lambda^3}}}$$

- Horn antenna

$$G = 20 \log\left(\frac{\pi D}{\lambda}\right) - 2.8 \quad \text{and} \quad \theta = \frac{255\lambda}{\pi D}$$

Task 4 - Calculating antenna gains for transmitter

By rearranging and substituting the equations from task 3, I can implement these into my code in order to calculate the gains for all three antennae. In order to then establish these in an array, I can use an if statement within a for loop, then combine all three values into an array:

```
for i = 1:length(P_Tss)
    if i == 1
        G_para = 44.3 - 10*log10(theta_bw(1)^2);
    elseif i == 2
        G_helix = 10.3 + 10*log10(2704/(theta_bw(2)^2));
    elseif i == 3
        G_horn = 20*log10(255/theta_bw(3)) - 2.8;
    end
end
```

% Uses first value from P_Tss
% array
% Calculates gain for parabolic
% antenna [dB]
% Uses second value from P_Tss
% array
% Calculates gain for helix
% antenna [dB]
% Uses third value from P_Tss
% array
% Calculates gain for horn
% antenna [dB]

```

end
end

G_array = [G_para, G_helix, G_horn];           % Groups results of for loop
                                              % into gain array

```

Task 5 - Calculating EIRP for transmitter

Using the respective values from the antenna gain array calculated in task 4 and the link budget equation from the lecture slides, I can calculate the effective isotropic radiated power (in its decibel form) of the system for each antenna type:

$EIRP = 10 \log_{10}(P_T G_T)$, however, since G_T is already in decibels but P_T is still in Watts, this equation can actually be written as:

$$EIRP = 10 \log_{10}(P_T) + G_T$$

```

EIRP_para  = 10*log10(P_Tss(1)) + G_para;           % EIRP for parabolic antenna
                                                    % [dBW]
EIRP_helix = 10*log10(P_Tss(2)) + G_helix;           % EIRP for helix antenna [dBW]
EIRP_horn  = 10*log10(P_Tss(3)) + G_horn;           % EIRP for horn antenna [dBW]

```

Making the output of this task an array:

```

EIRP_array = [EIRP_para, EIRP_helix, EIRP_horn];

```

Task 6 - Calculating gain for receiver

The tutorial states that the ground station antenna is parabolic and we are given the antenna diameter in Table 1. Using this information, I can calculate the antenna gain for the receiver in decibels using Equation 8.20 on page 178 of the course notes:

$G = 17.8 + 20 \log D + 20 \log f$, where D is the antenna diameter in m and f is the downlink frequency in GHz.

```

G_r = 17.8 + 20*log10(D_gs) + 20*log10(v_d);       % Gain of receiver [dB]

```

Task 7 - Calculating receiver gain-to-noise density ratio

GRN0R is given in the lecture slides in its decibel form as:

$GRN0R = 10 \log_{10} \left(\frac{G_R}{kT_R} \right)$, where k is the Boltzmann constant. In this equation, only the G_R term is in decibels.

Therefore, I will use:

$$GRN0R = G_R - (10 \log_{10} k + 10 \log_{10} T_R)$$

Inputting this into my code using the variables that I have defined previously gives:

```
k      = 1.38*10^-23;           % Boltzmann constant [J/K]
GRN0R = G_r - (10*log10(k) + (10*log10(T_Rgs))); % Receiver gain-to-noise density
                                           % ratio [dBW^-1Hz]
```

Task 8 - Defining slant range

Before free space loss is calculated, I must define the slant range for the given altitude range. In the best case scenario for a communications link, the transmitter would be directly above the receiver, meaning the signal would only need to travel a distance equal to the altitude of the satellite's orbit. I cannot assume that this spacecraft is always at this shortest distance from the ground station, therefore, using the slant range defines the worst case scenario. This is when the satellite is at the horizon when viewed from the terrestrial location and either first visible in the sky when rising or about to move out of direct line of sight when descending.

The tutorial brief states that h ranges from 250km to 2,500km:

```
h_slanrange = linspace(250000,2500000,1000); % Creates an array that covers
                                                % the given range of altitudes
S            = sqrt((R_E + h_slanrange).^2 - R_E^2); % Slant range (worst case
                                                % scenario) [m]
```

Task 9 - Calculating free space loss

FSL is given in the lecture slides in its decibel form as:

$$FSL = 20 \log_{10} \left(\frac{4\pi S}{\lambda} \right)$$

I will use the equation $\lambda = \frac{c}{f}$, where c is the speed of light in a vacuum and f is the frequency of the downlink in Hertz.

```
c      = 3*10^8;           % Speed of light in a vacuum
                           % [m/s]
lambda = c/(v_d*10^9);     % Wavelength of signal (convert
                           % frequency to Hz) [m]
L_FS   = 20*log10((4*pi*S)/lambda); % Free space loss in decibel
                           % form across altitude range
```

Task 10 - Estimating CN0R

The carrier to noise density ratio in its decibel form is given in the lectures slides as:

$$CN0R = 10 \log_{10} \left(\frac{C}{N_0} \right)$$

This equation is in terms of EIRP, GRN0R and free space loss, which have all been calculated in previous tasks. It also includes L_{add} which are the additional losses (does not include implementation losses), which is given in Table 2 of the tutorial. This value is already in decibels, therefore does not need converting like the other terms in the link budget equation.

The complete link budget equation in decibel form to calculate the carrier to noise density ratio is:

$$CN0R = EIRP + GRN0R - FSL - AL$$

I can now use the values calculated in previous tasks to calculate CN0R for all three antennas:

```

CN0R_para = EIRP_para + GRN0R - L_FS - L_add;           % Carrier to noise density ratio
                                                         % for parabolic antenna [dBHz]
CN0R_helix = EIRP_helix + GRN0R - L_FS - L_add;         % Carrier to noise density ratio
                                                         % for helix antenna [dBHz]
CN0R_horn = EIRP_horn + GRN0R - L_FS - L_add;           % Carrier to noise density ratio
                                                         % for horn antenna [dBHz]

```

Creating an array of these values across the range of worst case communications distances will make determining the link margin easier later on:

```

CN0R_array = [CN0R_para; CN0R_helix; CN0R_horn];

```

Task 11 - Calculating EBN0

The equation for energy per bit-to-noise density ratio is given in the lectures slides in decibel form as:

$$10 \log_{10} \left(\frac{E_b}{N_0} \right) = 10 \log_{10} \left(\frac{C}{N_0} \right) - 10 \log_{10}(R_b)$$

I already have the EBN0 and CN0R terms in the correct units, therefore only need to convert the data rate units. Carrier to noise density ratio was calculated in task 10 and data transmission rate (R_b) is given in Table 2 of the tutorial.

Creating the array:

```

EBN0_array = CN0R_array - (10*log10(R_b));             % Energy per bit to noise ratio

```

Task 12 - Calculating probability of error

The probability of error for a single bit indicates the link quality and is given in the tutorial as:

$$P_e = \frac{1}{2} e^{-10 \left(\frac{E_b}{N_0} \right)_{BER,est} \times \frac{1}{10}}$$

I have chosen to define my range of $\frac{E_b}{N_0}$ values as 0 - 15 dB:

```
EBN0_BER = linspace(0,15,1000); % Defines range of possible EBN0
                                     % values able to provide
                                     % expected performance
                                     % requirements [dB]
P_e       = 0.5*exp(-10.^(EBN0_BER*0.1)); % Creates an array for
                                     % probability of error for a
                                     % single bit
```

Task 13 - Obtaining modulation scheme performance

In order to find the $\frac{E_b}{N_0}$ value that is best suited for the DPSK modulation scheme, we must interpolate

between EBN0 and P_e values calculated in previous tasks:

```
MS_p = interp1(P_e, EBN0_BER, BER); % Modulation scheme performance
                                     % outputs a single value of
                                     % EBN0_BER which is best suited
                                     % to scheme [dB]
```

Task 14 - Determining link margin

In order to calculate the final link margin to see whether it is likely that this link will work, I will use the equation:

$$\text{Link margin} = \left(\frac{E_b}{N_0} \right)_{dB} - \left(\frac{E_b}{N_0} \right)_{BER(dB)} - (L_{imp})$$

Creating a final array containing the link margin for all antenna systems across the range of communications altitudes:

```
Linkmargin = EBN0_array - MS_p - L_i;
```

```
% Resultant link margin [dB]
```

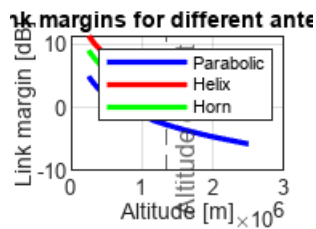
Task 15 - Displaying link margin vs altitude

Displaying all three downlink margins for different antennae on the same graph will allow for direct comparison:

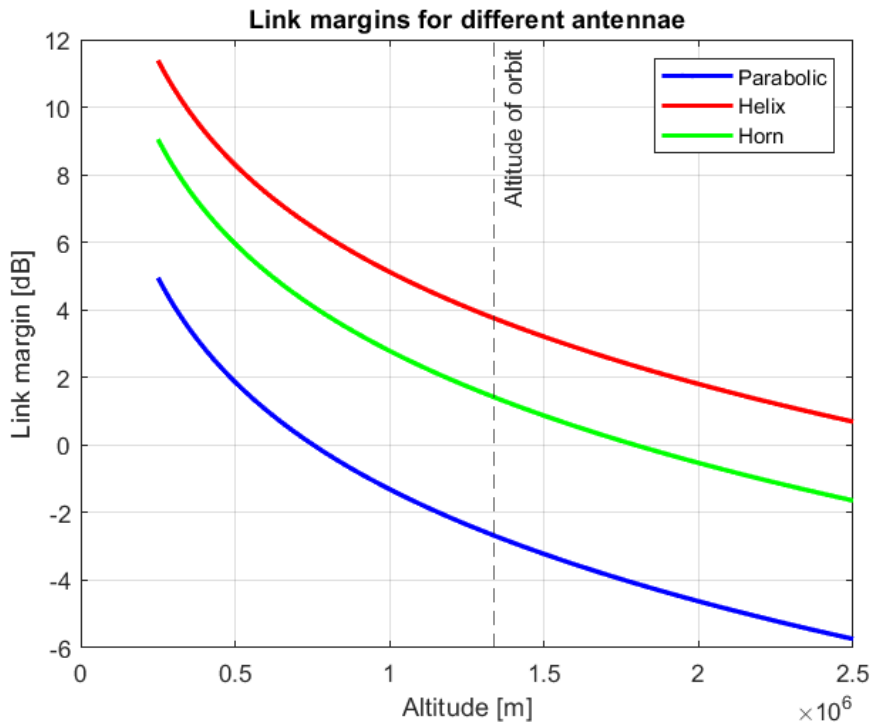
```
figure;
plot(h_slantrange, Linkmargin(1, :), 'b', 'Linewidth', 2);
hold on;
plot(h_slantrange, Linkmargin(2, :), 'r', 'Linewidth', 2);
hold on;
plot(h_slantrange, Linkmargin(3, :), 'g', 'Linewidth', 2);

title('Link margins for different antennae');
xlabel('Altitude [m]');
ylabel('Link margin [dB]');
legend('Parabolic', 'Helix', 'Horn');
grid on;

xline(1336*10^3, '--', {'Altitude of orbit'}, 'HandleVisibility', 'off'); % Actual planned
```



```
% orbital
% altitude of
% Sentinel-6B
```



Discussion

1. Based on my analysis, only the helix and horn antenna systems are suitable for use in the Sentinel-6B mission for the chosen orbital altitude, as their link margin lines intersect the vertical line on the graph at a positive link margin value. This means that it is likely that this link will always work for those two antennae but it is less likely that the parabolic one will, since its link margin is negative at Sentinel-6B's orbital altitude.
2. I will not be choosing the parabolic antenna because its mass is much greater than the other two which could make the mission inefficient, as it might require more fuel to launch the spacecraft compared to the other two. The parabolic antenna also has the worst link margin, meaning it would perform the worst out of the three across the range of altitudes. Although the horn antenna's mass is 1/6 of the helix antenna's mass, its noise is higher which would degrade the received signal. The helix antenna has the highest transmission power, which is desirable for this mission, since the signal will need to travel through the Earth's atmosphere all the way to the orbital altitude. The bandwidths for all three antennae systems were very similar and had a small range, so I did not include this in my consideration. Therefore, I would choose the helix antenna system to be installed on the Sentinel-6B mission due to many factors including the fact that it has the best link margin. Another reason why I would choose the helix antenna is because its frequency band is S. This is a lower band, which tends to use helix antennas. Horn antennas tend to be used for higher frequency bands because they have lower losses at higher frequencies and are more simple to construct.
3. I would reduce the ground station noise temperature by a factor of 2. This would not only increase the link budget, it would also increase the signal to noise ratio which would make the detection of the signal more reliable. It would increase the sensitivity of the ground station, allowing it to process the received signal even if it is weak, which would also allow communication for higher altitudes in case the spacecraft

would need to go into a larger orbit. A lower noise temperature would also enable the system to operate at higher data rates and it would reduce error rates.

AERO21111 Space Systems - MATLAB Tutorial 4

Attitude Determination and Control Systems

Task 1 - Defining inputs and physical constants

Creating the list of parameters needed using Table 1 from the tutorial:

```
R_E      = 6371*10^3;           % Radius of Earth [m]
mu       = 3.986*10^14;         % Standard gravitational parameter of Earth
                                   % [m^3*s^-2]
hTorque  = [1000000 1336000 2000000]; % Array of altitudes [m]
I_ZZ     = 40;                  % Mass moment of inertia about the z-axis
                                   % [kg*m^2]
I_YY     = 25;                  % Mass moment of inertia about the y-axis
                                   % [kg*m^2]
A_cross  = 3.35;                % Cross sectional area of the satellite [m^2]
delta_d  = 0.5;                 % Distance from centre of mass to centre of
                                   % aerodynamic pressure [m]
rho      = [3*10^-15 8.5*10^-16 7*10^-17]; % Array of density [kg*m^-3]
C_d      = 2.2;                 % Coefficient of drag
```

Tasks 2 to 8 - Calculating torques due to gravity and drag

In this tutorial, the torque due to gravity depends on four inputs:

- Mass moment of inertia about the z-axis (I_{ZZ})
- Mass moment of inertia about the y-axis (I_{YY})
- Radius of orbit (R)
- Standard gravitational parameter of Earth (μ)

These have all been defined in task 1, except for the radius of orbit, which is the sum of the radius of the Earth and altitude. I will code for this using my altitude array, creating a radius of orbit array:

```
R_o = R_E + hTorque;           % Radius of orbit [m]
```

The torque due to drag, as given in task 7, depends on five inputs:

- Array of density (ρ_{ADCS})
- Coefficient of drag (C_d)
- Cross sectional area of the satellite (A_{cross})
- Orbital velocity (V)
- Distance from centre of mass to centre of aerodynamic pressure (Δ_d)

These have also all been defined in task 1, except for orbital velocity, which is given as Equation 3.8 on page 44 of the course notes as:

$$v = \sqrt{\frac{\mu_{\oplus}}{r}}, \text{ where } r \text{ is the radius of the orbit.}$$

Inputting this into my code, taking the range of altitudes into consideration gives:

```
V = sqrt(mu./(R_E + hTorque)); % Velocity of orbit [m/s]
```

Then, using these inputs and the functions for both torques defined at the end of this code, I can create a for loop that can output both torques for any value within the altitude array defined.

```
for i = 1:length(hTorque)
    T_G(i) = calculateT_G(I_ZZ,I_YY,R_o(i),mu); % Uses function for torque due
                                                % to gravity
    T_D(i) = calculateT_D(rho(i),C_d,A_cross,V(i),delta_d); % Uses function for torque due
                                                            % to drag
end
```

Task 9 - Calculating orbit period

This is Equation 3.9 on page 44 of the course notes:

$$\tau = 2\pi \sqrt{\frac{r^3}{\mu}}$$

Inputting this equation into the code using vector operations gives:

```
tau = 2*pi*sqrt(R_o.^3/mu); % Orbit period [s]
```

Tasks 10 to 12 - Calculating total torque over time and momentum storage

In order to analyse the variation of torque over time, a function relating torque and time is required. This is given in the tutorial as:

$$f(t) = \cos\left(\frac{t}{\tau} 2\pi\right) T_G + T_D$$

Then, using the integral function, I will calculate the momentum storage over one orbit, which will be displayed on each graph for different altitudes.

```
for j = 1:length(hTorque)
    torqueovertime_tot = @(t_tot) cos((t_tot*2*pi)./tau(j)).*T_G(j) + T_D(j); % Creates a
                                                                                   % temporary
                                                                                   % variable t and
                                                                                   % calculates
                                                                                   % change in
                                                                                   % torque over
                                                                                   % time
    Mom_storage(j) = integral(torqueovertime_tot,0,tau(j)); % Integrates
                                                                                   % above function
                                                                                   % and calculates
                                                                                   % angular
                                                                                   % momentum
                                                                                   % [kgm^2/s]

    figure
    H = ['Angular momentum over one orbit at ', num2str(hTorque(j)), ' m is ',...
        num2str(Mom_storage(j)), ' kgm^2/s'];
    disp(H) % Displays value
                                                                                   % of momentum
                                                                                   % storage for
                                                                                   % all graphs
    x = 0:100:tau(j); % Defines limits
                                                                                   % and step size
                                                                                   % of x axis
```

Task 13 - Plotting the results

Lastly, in order to indicate the area that I have integrated over, I will use the area function in my code instead of the plot function:

```
area(x,torqueovertime_tot(x)); % Plots torque
                                                                                   % gradients
                                                                                   % against orbit
                                                                                   % period and
                                                                                   % shows area
                                                                                   % integrated
                                                                                   % over

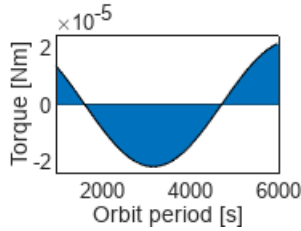
    title = ['Torque variation over orbit period at altitude ', num2str(hTorque(j)), ' m'];
    disp(title) % Displays title
                                                                                   % for all graphs

    xlabel('Orbit period [s]);
```

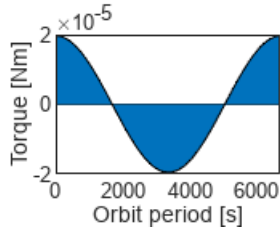


```
ylabel('Torque [Nm]');  
end
```

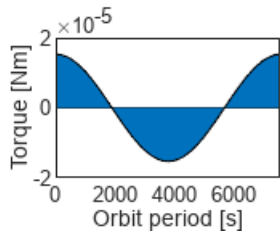
Angular momentum over one orbit at 1000000 m is 0.0018825 kgm²/s
Torque variation over orbit period at altitude 1000000 m



Angular momentum over one orbit at 1336000 m is 0.0005454 kgm²/s
Torque variation over orbit period at altitude 1336000 m



Angular momentum over one orbit at 2000000 m is 4.681e-05 kgm²/s
Torque variation over orbit period at altitude 2000000 m



Discussion

1. An ADCS system could be added in order to stabilise the spacecraft, which would reduce the roll angle. The distribution of mass could change in order to make the mass moment of inertia about the z and y axes the same. Adding an ADCS system would increase the cost of the mission and also its mass which would be undesirable for just a small difference in torque variation. Both of these would either decrease the torque due to gravity or make it equal to zero, which would in turn stop or reduce the variation of torque across the orbit. Another way to achieve this is by decreasing the torque due to drag. This can be done by reducing the cross sectional area of the spacecraft, ensuring all components of the satellite are mounted in a way that makes the centre of mass as close to the centre of aerodynamic pressure as possible or change the shape of the satellite to make it more aerodynamic which would reduce the coefficient of drag. This would be difficult to implement in a real system because there would need to be a significant change in the design of the spacecraft, which could take a while and be technically challenging.
2. In this model, the satellite is in LEO, meaning the effect of the Earth's magnetic field would be more significant in inducing a torque compared to higher altitude orbits. Also, there is still some atmosphere in LEO, meaning compared to higher altitude orbits, the effects of solar radiation are more significant.

Because of this, I do not think that it is reasonable to neglect the disturbance torques due to magnetic effects and solar radiation.

3. A suitable attitude control system for this spacecraft would be the MTQ800 magnetorquer. Its mass is 395g and power consumption is 3W. Magnetorquers do not consume fuel and do not wear out due to not having moving parts, meaning they are ideal for long term missions. They are lightweight and do not consume much power compared to other ADCS systems. They are also very suitable for use in LEO because they depend on an external magnetic field like Earth's.

AERO21111 Space Systems - MATLAB Tutorial 5

Power and Mass Budget Analysis

Initial Tasks

Task 1 - Estimating approximate mission lifetime

I know from the course notes that the number of charge/discharge cycles of a battery is equal to the number of orbits over the mission lifetime. Orbital period was calculated in tutorial 2 and the estimated number of recharge cycles (before degradation) is given in this tutorial in Table 1:

CC = 25000;	% Estimated number of recharge cycles
N_o = CC;	% Number of orbits is equal to number of
	% charge cycles

The total mission lifetime is the number of orbits multiplied by the orbital period:

ML = T_o*N_o;	% Mission lifetime estimate [s]
---------------	---------------------------------

The output is 1.6834×10^8 seconds, which is 5.338 years. I have chosen to round this down to a mission lifetime estimate of 5 years because it is not close enough to 6 years to realistically round it up. If it is rounded up, there is a risk that the battery degrades before it reaches 6 years.

Task 2 - Estimating overall power demand

Using Table 2, the eclipse duty cycle and my power estimation demands from previous tutorials, the overall worst case power budget can be estimated:

- Payload = 0 W (turned off in eclipse)

- Thermal subsystem = 150 W (fully operational)
- Heater = N/A (I did not use one in tutorial 2)
- Communications subsystem = 120 W (won't be processing any data, only requires half power)
- Transmission power = 90 W (I chose a helix antenna for the space segment in tutorial 3)
- ADCS subsystem = 55 W (not needed for payload pointing, only requires half power)
- Reaction wheels = 3 W (I used a magnetorquer instead of a reaction wheel)
- All other subsystems = 75 W (placed in standby mode, only require half power)

Total power required = 493 W

```
Pay_Pow_e    = 0;           % Power required by payload in eclipse [W]
Therm_Pow_e  = 150;         % Power required by thermal subsystem in
                             % eclipse [W]
Comm_Pow_e   = 120;         % Power required by communications subsystem
                             % in eclipse [W]
Trans_Pow_e  = 90;          % Power required by transmission in eclipse
                             % [W]
ADCS_Pow_e   = 55;          % Power required by ADCS subsystem in eclipse
                             % [W]
Magn_Pow_e   = 3;           % Power required by magnetorquer in eclipse
                             % [W]
Other_Pow_e  = 75;          % Power required by all other subsystems in
                             % eclipse [W]

P_e          = Pay_Pow_e + Therm_Pow_e + Comm_Pow_e + Trans_Pow_e + ADCS_Pow_e + ...
               Magn_Pow_e + Other_Pow_e;      % Total power required in eclipse [W]
```

Task 3 - Calculating size of energy storage system

There is an equation for total battery capacity in Ampere-Hours on page 167 of the course notes:

$$C_B = \frac{P_{eclipse} t_e}{DOD V_B},$$

where t_e is the length of eclipse period in hours, $P_{eclipse}$ is the power required during eclipse, DOD is the depth of discharge (given in Table 1) and V_B is the bus voltage (given in initial question).

```
t_e_hr = t_e/(3600);       % Eclipse time [hrs]
DOD     = 0.5;              % Operational depth of discharge
V_B     = 18;               % Output voltage of the chosen satellite bus
                             % [V]
C_B     = (P_e*t_e_hr)/(DOD*V_B); % Total battery capacity [Ah]
```

Task 4 - Estimating total energy of storage system

The equation in the course notes gives the total energy for the storage system in Watt-Hours as:

$$\epsilon = C_B \times V_B$$

E_stored = C_B*V_B;	% Total energy of storage system [Wh]
---------------------	---------------------------------------

Task 5 - Estimating total mass of battery

The total mass of the battery required to maintain power on the Sentinel-6B spacecraft is given in the course notes as:

$$m_{batt} = \epsilon \div \bar{\epsilon} ,$$

where $\bar{\epsilon}$ is the energy density of the battery (given in Table 1).

E_density = 120;	% Energy density [Wh/kg]
m_batt = E_stored/E_density;	% Total mass of the battery [kg]

This gives 4.7605 kg, which I will round up to 5 kg for the total mass of the battery.

Follow-On Tasks

Task 1 - Defining constants

cell_eff = 0.2;	% Cell efficiency
D_yr = 0.05;	% Degradation per year from initial baseline % performance
pack_eff = 0.9;	% Packing efficiency
theta_i = 40;	% Average incident Sun angle [degrees]
V_A = 24;	% Array output voltage [V]

Task 2 - Calculating battery charging rate

In the course notes, the battery charging rate is given as:

$$R_{ch} = \frac{DOD C_B}{t_s} ,$$

where t_s is the time in sunlight in hours.

t_s_hr = t_s/(3600);	% Time in sunlight [hrs]
R_ch = (DOD*C_B)/t_s_hr;	% Battery charging rate [Amps]

Task 3 - Calculating total power needed during sunlight

For this power calculation, I can use the values straight from Table 2 because all subsystems will be at full power during the Sun phase:

Pay_Pow_s = 350;	% Power required by payload in eclipse [W]
Therm_Pow_s = 150;	% Power required by thermal subsystem in eclipse [W]
Comm_Pow_s = 240;	% Power required by communications subsystem in eclipse [W]
Trans_Pow_s = 90;	% Power required by transmission in eclipse [W]
ADCS_Pow_s = 110;	% Power required by ADCS subsystem in eclipse [W]
Magn_Pow_s = 3;	% Power required by magnetorquer in eclipse [W]
Other_Pow_s = 150;	% Power required by all other subsystems in eclipse [W]
P_s = Pay_Pow_s + Therm_Pow_s + Comm_Pow_s + Trans_Pow_s + ADCS_Pow_s + ... Magn_Pow_s + Other_Pow_s;	% Total power required in Sun phase [W]

The power required to enable all services during the Sun phase is 1093 W.

In order to calculate the total power required, I must add the power required to charge the onboard batteries to this value. This is given in an equation in the course notes:

$$P_{eol} = P_{sunlight} + (R_{ch} \times V_A) ,$$

where V_A is the array voltage (given in Table 3).

P_tot = P_s + (R_ch*V_A);	% Total power array must provide when illuminated [W]
---------------------------	---

This gives an output of 1133.7 W. Rounded up to the nearest whole Watt gives a final answer of 1134 W.

Task 4 - Estimating area of solar panels

The equation for the area of solar panels required to deliver the required power input when illuminated is given in the course notes as:

$$A_{array} = \frac{P_{eol}}{q_s \eta \eta_p (1 - D) \cos \theta} ,$$

where η , η_p , D and θ are given in Table 3 and q_s is given in the question:

```
q_s      = 1400;                                % Solar insolation
                                                % (intensity) [W/m^2]
A_array = P_tot/(q_s*cell_eff*pack_eff*(1-D_yn)*cosd(theta_i)); % Area of array of solar
                                                % panels [m^2]
```

This gives an output of 6.182 m^2 . Rounded up to the nearest whole square metre gives a final answer of 7 m^2 .

Task 5 - Estimating mass of solar array

We know that for Sentinel-6A, its solar panels have an area of 15 m^2 and weigh 150 kg. For Sentinel-6B, its solar panels have an area of 6.182 m^2 , so to find its mass, we can use the mass/area relationship:

```
m_array_6A = 150;                                % Mass of Sentinel-6A
                                                % solar array [kg]
A_array_6A = 15;                                % Area of Sentinel-6A
                                                % solar array [m^2]
m_array_6B = (m_array_6A/A_array_6A)*A_array;    % Mass of Sentinel-6B
                                                % solar array [kg]
```

This gives an output of 61.82 kg. Rounded up to the nearest whole kilogram gives a final answer of 62 kg.

Discussion

1. The following is the completed power budget table:

Sub-System Element	Sentinel 6-A [W]	Sentinel 6-B [W]
Payload	350	350
Thermal Subsystem	150	150
Heater	N/A	N/A
Communications Subsystem	240	240
Transmission Power	30	90
ADCS Subsystem	110	110
Reaction Wheels	N/A	3
All Other Subsystems	150	150
Total	1030	1134

2. The following is a completed mass budget table:

Sub-System Element	Sentinel 6-A [kg]	Sentinel 6-B [kg]
Payload and Structure	465	465
Thrusters	50	50
Manouvering Fuel	130	130
De-Orbit Fuel	<i>N/A</i>	780
Thermal Subsystem	40	40
Heater	<i>N/A</i>	<i>N/A</i>
Communications Subsystem	70	70
Transmission Antenna	30	12
ADCS Subsystem	60	60
Reaction Wheels	<i>N/A</i>	1
Power Distribution System	50	50
Power Regulation and Control System	70	70
Energy Storage System	15	5
Solar Array	150	62
Total	1130	1795

3. In this tutorial, I have calculated the power and mass budgets of Sentinel-6B, which included my choices of subsystems from previous tutorials. I chose not to use a heater because that would require extra time, weight and money for integration, it would require maintenance for the entire mission lifetime. It could also change the centre of gravity of the spacecraft which would affect things like torque variation and this would change many paramteres that affect the trajectory of Sentinel-6B. I already justified my choice of transmission antenna in tutorial 3 and looking at discussion points 1 and 2 above, it is clear that compared to Sentinel-6A, Sentinel-6B's antenna has triple the transmission power with less than half the weight. Overall, Sentinel-6B has a slightly higher power budget even though it is much heavier than Sentinel-6A. This is because it contains more subsystems and is able to regulate disturbance torques, whereas Sentinel-6A cannot. Sentinel-6B's mass is almost exactly the same as the value of total mass I estimated in tutorial 1 (1800 kg) and most of it is due to the de-orbit fuel. Sentinel-6A does not have this element in its mass budget, meaning at the end of its useful lifetime, it will not be transferred to a graveyard orbit which is dangerous. However, Sentinel-6B will be transferred to a graveyard orbit to reduce the chance of colliding with operational spacecraft after it has become space debris.

4. Throughout these five tutorials, I have performed propulsion sensitivity analysis, investigated effects of different thermal designs, considered several communications techniques, analysed ADCS and estimated power and mass budgets for the Sentinel-6B mission. I now have an enhanced understanding of the various systems on board a spacecraft and how they affect its trajectory and mission lifetime. I believe that my assumptions throughout the tutorials have been reasonable because in many cases, I have considered the worst case scenario, which would prepare the mission design team thoroughly.

Creating the functions:

The equation for torque due to gravity can be found on page 189 of the course notes as Equation 9.6:

$$T_G = \left(\frac{3\mu}{2R^3} \right) |I_{ZZ} - I_{YY}| \sin(2\theta), \text{ where } \theta \text{ is the roll angle.}$$

Since we are trying to find the maximum torque, we must consider the worst case scenario for the roll angle.

The maximum torque occurs when $\sin(2\theta) = 1$, therefore $\theta = 45^\circ$:

```
function [T_G] = calculateT_G(I_ZZ,I_YY,R_o,mu)
T_G = ((3*mu)/(2*R_o^3))*abs(I_ZZ - I_YY)*sind(2*45);
end
```

The equation for torque due to drag is given in the tutorial as:

$$T_D = \frac{1}{2} \rho C_d A V^2 (C_{pa} - C_g)$$

```
function [T_D] = calculateT_D(rho,C_d,A_cross,V,delta_d)
T_D = 0.5*rho*C_d*A_cross.*V.^2*delta_d;
end
```